



BÀI 03: LỚP VÀ CÁC THÀNH PHẦN CỦA LỚP ĐỐI TƯỢNG

- ❑ Định nghĩa lớp các đối tượng
- ❑ Các thành phần của lớp
- ❑ Phạm vi truy cập
- ❑ Toán tử tạo lập



Định nghĩa lớp các đối tượng

```
[<Phạm vi hoặc kiểm soát truy nhập>] class <Tên lớp>  
    [extends <Tên lớp cha>] [implements <Tên giao diện>]  
{  
    <Các thành phần của lớp>  
}
```

Trong đó:

- + **class, extends, implements** là các từ khoá.
- + Những phần trong cặp [và] là tùy chọn.



Định nghĩa lớp các đối tượng

```
import java.io.*;
class Person{
    private String hoTen;
    private String diaChi;
    private int namSinh;

    Person(){}
    Person(String ht, String dc, int ns){
        /*Person(String hoTen, String diaChi, int namSinh){
    Person(String ht, String dc){
    public String nhapXau(){
    public int doiInt(String st){
    public void nhapTT(){
    public void hienThiTT(){
}
```

```
public class NhanVien extends Person{
    private int tongLuong;

    NhanVien(){}
    NhanVien(String ht, String dc, int ns, int tl){
    public void nhapTT(){
    public void hienThiTT(){
    public static void main(String arg[]){
        NhanVien nv=new NhanVien();
        nv.nhapTT();
        nv.hienThiTT();
    }
}
```



Định nghĩa hàm thành phần

[<P.Vi hoặc TT K.Soát truy nhập>]<Kiểu trả lại> <Tên hàm >(
[<Danh sách tham biến hình thức>]) [<Mệnh đề **throws**>]
{
 <Nội dung của hàm>
}

Trong đó:

+ <Kiểu trả lại> có thể là kiểu nguyên thủy, kiểu lớp hoặc không có giá trị trả lại (kiểu *void*).

+ <Danh sách tham biến hình thức> bao gồm dãy các tham biến (kiểu và tên) phân cách với nhau bởi dấu phẩy



Ví dụ 1

```
public class Demo{
    public static void main(String args[]){
        if (args.length == 0) return; //Hàm kiểu void có thể sử dụng return;
        output(checkValue(args.length));
    }
    static int sum(int m, int n){ // Hàm kiểu int; tính tổng giá trị của các tham số
        int kq=m+n;
        System.out.println("Sum =" +kq);
        return kq;
    }
    static int checkValue(int i){ // Hàm khác kiểu void phải sử dụng return để trả
        if (i > 3) return 1; // lại giá trị.
        else return 2;
    }
}
```



Ví dụ 2

```
class Person{
    private String hoTen;
    private String diaChi;
    private int namSinh;
    Person(){}
    Person(String hoTen, String diaChi, int namSinh){
        this.hoTen=hoTen;
        this.diaChi=diaChi;
        this.namSinh=namSinh;
    }
    public void nhapTT(){
        Scanner sc=new Scanner(System.in);
        System.out.print("Ho ten :"); hoTen=sc.nextLine();
        System.out.print("Dia chi :"); diaChi=sc.nextLine();
        System.out.print("Nam sinh :"); namSinh=sc.nextInt();
    }
    public void hienThiTT(){
        System.out.println("Ho ten =" + hoTen);
        System.out.println("Dia chi =" + diaChi);
        System.out.println("Nam sinh =" + namSinh);
    }
}
```



Ví dụ 2

```
public class NhanVien extends Person{
    private String chuyenMon;
    private int tongLuong;
    NhanVien(){
    NhanVien(String cm, int tl){
    NhanVien(String ht, String dc, int ns, String cm, int tl){
    public void nhapTT(){
        Scanner sc=new Scanner(System.in);
        super.nhapTT();
        System.out.print("Chuyen mon :"); chuyenMon=sc.nextLine();
        System.out.print("Tong luong :"); tongLuong=sc.nextInt();
    }
    public void hienThiTT(){
        super.hienThiTT();
        System.out.println("Chuyen Mon =" + chuyenMon);
        System.out.println("Tong luong =" + tongLuong);
    }
    public static void main(String arg[]){
}
```



Phạm vi truy cập

- ❑ *Phạm vi lớp của các thành phần*: Quyền truy nhập của chúng thường được xác định thông qua các *bổ ngữ (modifier)*: *public, protected, private*.
- ❑ *Phạm vi khối của các biến cục bộ (local)*: Tuân theo *Luật phạm vi*: một biến được khai báo ở trong một khối có phạm vi xác định bên trong khối và không xác định ở bên ngoài khối đó (Khối được bắt đầu bởi “{“, kết thúc bởi “}”)



Phạm vi truy cập

- ❑ **public:** Công cộng, công khai
- ❑ **protected:** Được bảo vệ
- ❑ **mặc định** (không sử dụng thêm bổ ngữ khi định nghĩa lớp)
- ❑ **private:** Sở hữu riêng
- ❑ **static:** Chung cho cả lớp; không phải riêng của đối tượng nào cả
- ❑ **final:** Sau lần gán giá trị đầu tiên sẽ trở thành hằng số
- ❑ **abstract:** Thành phần trừu tượng
- ❑ Một số từ khóa khác: synchronized, native, transient, volatile



Một số lưu ý

Modifier	Same class	Same package	Subclass	Universe
private	Yes			
package (default)	Yes	Yes		
protected	Yes	Yes	Yes	
public	Yes	Yes	Yes	Yes



Một số lưu ý

- ❑ Từ khoá static
 - ❑ Ví dụ
- ❑ Từ khoá final
 - ❑ Ví dụ
- ❑ Từ khoá abstract
 - ❑ Ví dụ



Toán tử tạo lập đối tượng

Mục đích chính của toán tử tạo lập (*constructor*): Đặt các giá trị khởi tạo cho các đối tượng khi một đối tượng được tạo ra bằng toán tử *new*.

Khai báo:

```
[<Thuộc tính>] <Tên lớp> ([<danh sách tham biến>]) {  
    // Nội dung cần tạo lập  
}
```



Toán tử tạo lập đối tượng

- ❑ Toán tử tạo lập giống như các hàm thành phần và chỉ khác ở phần đầu của định nghĩa:
- ❑ <Thuộc tính> chỉ có thể sử dụng: *public*, *protected*, *private* hoặc mặc định như đã trình bày ở phần trên.
- ❑ <Tên lớp> luôn trùng với tên của lớp chứa toán tử đó.

Một số lưu ý đối với các toán tử tạo lập:

- ❑ Không sử dụng các bộ ngữ khác với toán tử tạo lập,
- ❑ Toán tử tạo lập là loại hàm đặc biệt không có kiểu trả lại, ngay cả kiểu *void* cũng không được sử dụng,
- ❑ Toán tử tạo lập chỉ sử dụng được với toán tử *new*.



Toán tử tạo lập mặc định

- ❑ Toán tử tạo lập mặc định (*default*) là toán tử tạo lập không có tham biến.

Cấu trúc: <Tên lớp> () { /* ... */ }

- ❑ Khi một lớp không định nghĩa một toán tử tạo lập nào cả thì hệ thống sẽ tự động cung cấp toán tử tạo lập mặc định không tường minh. Toán tử tạo lập mặc định không tường minh tương đương với dạng:

<Tên lớp> () { } // Không làm gì cả

- ❑ Lưu ý: Khi đã định nghĩa toán tử tạo lập thì hệ thống sẽ không cung cấp toán tử tạo lập mặc định không tường minh nữa.



Nạp chồng toán tử tạo lập

```
class BongDen{  
    // Biến thành phần  
        int soWatts;  
        boolean batTat;  
        String viTri;  
    // Định nghĩa toán tử tạo lập mặc định  
    BongDen(){ soWatts = 40; batTat = true; viTri = new String("XX"); }  
    // Định nghĩa toán tử tạo lập không mặc định  
        BongDen(int w, boolean s, String v){ soWatts = w; batTat = s;  
        viTri = new String(v);  
        }  
}
```



BÀI TẬP

Viết chương trình xây dựng lớp SoPhuc gồm:

Các thuộc tính riêng gồm: `phanThuc`, `phanAo` kiểu `double`

Các phương thức:

+ Các toán tử tạo lập : `SoPhuc()`, `SoPhuc(float pt, float pa)`

+ Phương thức nhập vào một số phức: `void nhapSoPhuc()`

+ Phương thức hiển thị một số phức: `void inSoPhuc()`

+ Phương thức cộng hai số phức : `SoPhuc congSoPhuc(SoPhuc sp)`

+ Phương thức nhân hai số phức: `SoPhuc nhanSoPhuc(SoPhuc sp)`

Cài đặt chương trình (lớp Main) nhập vào hai số phức A và B, sau đó tính số phức tổng $C1 = A.congSoPhuc(B)$, nhân $C = A.nhanSoPhuc(B)$ rồi hiển thị kết quả ra màn hình như sau:

Tổng: $(a + b.i) + (c + d.i) = (a + c) + (b + d).i$

Tích: $(a + b.i)(c + d.i) = (a.c - b.d) + (b.c + a.d).i$



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM